

Session 18 Live Demo Plan

Has Pop Music Gotten Sadder?

Dr. Sara Weston

June 3, 2026

A two-act worked example for Session 18. **Act 1** runs through a prepared longitudinal analysis on Spotify audio features; **Act 2** invites students to pick artists in class and analyzes them live. The slide deck still shows the GSS demo — this plan replaces it during live coding.

Data: `files/data/spotify_songs.csv` (TidyTuesday Jan 2020) — 32,833 tracks across 10,693 artists, 1957–2020.
Time budget: Act 1 ~ 15 min · Act 2 ~ 10 min · Total ~ 25 min.

Setup

Load packages, read the data, register the custom theme. Do this off-screen before class so the chunks below run quickly.

```
library(tidyverse)
library(here)
library(broom)

songs <- read_csv(here("files/data/spotify_songs.csv"))
```

Custom theme

A clean theme that matches the slide aesthetic — slate text, flat-UI palette, no minor gridlines, left-aligned bold titles. Registered globally so every plot inherits it.

```
psy410_palette <- c(
  "#3498db", # blue
  "#e74c3c", # red
  "#e67e22", # orange
  "#27ae60", # green
  "#9b59b6", # purple
  "#7f8c8d"  # gray
)

theme_psy410 <- function(base_size = 13) {
  theme_minimal(base_size = base_size) +
    theme(
      text           = element_text(color = "#2c3e50"),
      plot.title     = element_text(face = "bold",
                                     size = base_size + 3,
                                     margin = margin(b = 4)),
      plot.subtitle  = element_text(color = "#7f8c8d",
                                     size = base_size,
                                     margin = margin(b = 10)),
      plot.caption   = element_text(color = "#7f8c8d",
                                     size = base_size - 2),
```

```

    plot.title.position = "plot",
    axis.title          = element_text(color = "#2c3e50",
                                       size = base_size - 1),
    axis.text           = element_text(color = "#7f8c8d"),
    panel.grid.minor    = element_blank(),
    panel.grid.major    = element_line(color = "grey92"),
    legend.position     = "top",
    legend.title        = element_text(color = "#2c3e50",
                                       face = "bold",
                                       size = base_size - 1),
    legend.text         = element_text(color = "#2c3e50")
  )
}

theme_set(theme_psy410())
options(
  ggplot2.discrete.colour = psy410_palette,
  ggplot2.discrete.fill   = psy410_palette
)

```

With `theme_set()` and the `ggplot2.discrete.*` options, every plot in this session uses the theme and palette by default. You can drop + `theme_minimal()` and most `scale_color_manual()` calls from the plots below.

Act 1 — Has pop music gotten sadder?

Spotify scores every track on **valence** (0 = sad, 1 = happy). Has mean valence trended down across the last two decades? Does the answer differ by genre?

Step 1 — Get oriented

Say: “First move with any dataset: glimpse it.”

```
glimpse(songs)
```

Watch for: ~33k rows; audio features (valence, energy, danceability) as doubles between 0–1.

Step 2 — Check missing data

```

songs |>
  summarize(across(everything(), ~ sum(is.na(.x))))

```

Watch for: A few NAs in `track_name` / `album_name` / `release_year`. Audio features are all complete.

Step 3 — What genres are in here?

```
songs |> count(genre)
```

Watch for: Six even-ish buckets — edm, latin, pop, rap, r&b, rock (each ~5k tracks).

Step 4 — Quick clean

Drop NAs, dedupe tracks that appear on multiple playlists, restrict to 2000+ for solid year coverage.

```

songs_clean <- songs |>
  drop_na(release_year, valence) |>
  distinct(track_id, .keep_all = TRUE) |>
  filter(release_year >= 2000)

```

```
nrow(songs_clean)
```

Watch for: ~27,000 tracks remaining.

Step 5 — What does valence look like?

Say: “Univariate EDA — Session 10. What’s the shape of one variable?”

```
ggplot(songs_clean, aes(valence)) +  
  geom_histogram(binwidth = 0.05, fill = "#3498db", color = "white") +  
  labs(  
    title = "Distribution of musical valence",  
    subtitle = "0 = sad, 1 = happy",  
    x = "Valence", y = "Number of tracks"  
  )
```

Watch for: Surprisingly flat — songs span the full happy-to-sad spectrum.

Step 6 — Mean valence by year

```
year_valence <- songs_clean |>  
  group_by(release_year) |>  
  summarise(mean_valence = mean(valence), n = n(), .groups = "drop")  
  
year_valence
```

Step 7 — Plot the trend

```
ggplot(year_valence, aes(release_year, mean_valence)) +  
  geom_line(color = "#3498db", linewidth = 1.2) +  
  geom_point(color = "#3498db", size = 3) +  
  geom_smooth(method = "lm", se = TRUE, color = "#e74c3c") +  
  labs(  
    title = "Has pop music gotten sadder?",  
    subtitle = "Mean valence of top tracks by release year",  
    x = "Release year", y = "Mean valence (0 = sad, 1 = happy)"  
  )
```

Watch for: Clear downward drift — mean valence falls roughly 0.6 -> 0.45 across the period.

Step 8 — Is the trend uniform across genres?

```
songs_clean |>  
  group_by(release_year, genre) |>  
  summarise(mean_valence = mean(valence), .groups = "drop") |>  
  ggplot(aes(release_year, mean_valence, color = genre)) +  
  geom_line(linewidth = 1) +  
  labs(  
    title = "The sadness trend hits some genres harder than others",  
    x = "Release year", y = "Mean valence", color = "Genre"  
  )
```

Watch for: Latin and pop stay happier; rap and rock drop the most.

Step 9 — Quantify with regression

Say: “Monday we did correlation. Today we go one step further — a regression gives us a *slope*: how much does valence change per year?”

```
valence_model <- lm(valence ~ release_year, data = songs_clean)  
  
tidy(valence_model)
```

Watch for: release_year estimate ~ -0.009 per year — real, highly significant.

Step 10 — How well does the model fit?

```
glance(valence_model) |>
  select(r.squared, adj.r.squared, sigma, nobs)
```

Watch for: R^2 is tiny (~ 0.02). Year explains barely any variance — the trend is real but most variation is *between songs*, not across time. **Talking point:** statistical significance $\neq$ practical importance.

Step 11 — Add genre to the model

```
genre_model <- lm(valence ~ release_year + genre, data = songs_clean)
tidy(genre_model)
```

Watch for: Genre coefficients now visible (relative to the alphabetically-first level, edm). Latin and pop are positive; rap and rock are smaller. R^2 should jump.

Step 12 — Write it up with inline R

This is what the Quarto Results section looks like — every number is computed live.

```
## Results
```

```
We analyzed `r nrow(songs_clean)` tracks released between
`r min(songs_clean$release_year)` and `r max(songs_clean$release_year)`.
Mean valence was `r round(mean(songs_clean$valence), 2)`
(SD = `r round(sd(songs_clean$valence), 2)`).
```

```
Valence was negatively associated with release year
(b = `r round(coef(valence_model)[2], 4)` per year, p < .001),
consistent with the claim that popular music has trended away
from "happy" sounds over the past two decades.
```

Say: “Change the dataset, every number updates on re-render. That’s what reproducibility buys you.”

Act 2 — Your turn

Prompt the class: “Pick two artists you want to compare. Anyone, as long as you can spell their name.”

Take 2–3 names. Type them in live.

Step 13 — Filter to their picks

```
their_picks <- c("Taylor Swift", "Drake") # replace with student picks

picks <- songs |>
  filter(artist %in% their_picks)
```

Step 14 — How many tracks do we have for each?

```
picks |> count(artist)
```

Watch for: Could be 5 tracks or 50 — depends on the artist. If an artist has 0 rows, fall back to:

```
songs |> filter(str_detect(artist, "swift|drake"), ignore.case = TRUE) # try partial match
# or just say "not in the 2020 snapshot - we'd need a fresh pull"
```

Step 15 — Whose songs are happier?

```
ggplot(picks, aes(artist, valence, fill = artist)) +  
  geom_boxplot(alpha = 0.7) +  
  labs(  
    title = "Valence by artist",  
    x = NULL, y = "Valence (0 = sad, 1 = happy)"  
  ) +  
  theme(legend.position = "none")
```

Step 16 — The mood map

Say: “Spotify’s ‘mood’ is really two axes — happy vs sad AND high energy vs low energy. Together they make four quadrants: party songs (top right), angry songs (bottom right), chill happy (top left), sad ballads (bottom left).”

```
ggplot(picks, aes(energy, valence, color = artist)) +  
  geom_point(size = 3, alpha = 0.7) +  
  geom_hline(yintercept = 0.5, linetype = "dashed", color = "grey50") +  
  geom_vline(xintercept = 0.5, linetype = "dashed", color = "grey50") +  
  labs(  
    title = "The mood map",  
    x = "Energy (calm -> intense)", y = "Valence (sad -> happy)"  
  )
```

Watch for: Different artists cluster in different quadrants — Drake bottom-left (sad/medium-energy), Taylor Swift more spread out.

Step 17 — Has each artist’s mood shifted over time?

```
picks |>  
  group_by(artist, release_year) |>  
  summarise(mean_valence = mean(valence), .groups = "drop") |>  
  ggplot(aes(release_year, mean_valence, color = artist)) +  
  geom_line(linewidth = 1.2) +  
  geom_point(size = 2) +  
  labs(  
    title = "How each artist's valence has shifted",  
    x = "Release year", y = "Mean valence"  
  )
```

Watch for: Either flat or a clear arc. Either is interesting — “no change” is also a finding.

If time runs short

- **Cut Act 1 down** to Steps 4, 6, 7, 9, 12 (clean -> yearly average -> trend plot -> regression -> writeup).
- **Skip Act 2 entirely** — the lecture still has a complete worked example.
- **Drop the genre model (Step 11)** — keeps the regression message simpler.

If something breaks live

- **Code doesn’t run:** project the slides’ GSS demo instead — same workflow, different data.
- **Student picks an artist with 0 rows:** use `str_detect()` for a partial match, or move on with one of the artists that did match.
- **Quarto inline-R demo fails:** describe it verbally and show the rendered prose only.

Functions touched

`read_csv`, `glimpse`, `summarize`, `across`, `is.na`, `count`, `drop_na`, `distinct`, `filter`, `group_by`, `summarise`, `mean`, `n`, `mutate`, `lm`, `tidy`, `glance`, `coef`, `select`, `nrow`, `min`, `max`, `round`, `ggplot`, `geom_histogram`, `geom_line`, `geom_point`,

`geom_smooth`, `geom_boxplot`, `geom_hline`, `geom_vline`, `labs`, `theme_minimal`, `theme`, `str_detect`, inline R in Quarto.